Lean Formalization Seminar

Lean is a functional programming language with a community-developed library of formalized mathematics called *Mathlib* that is used to express mathematical theorems in a format that can be verified programmatically. The Mathlib library provides the semantic structure and definitions for proofs and statements, while the Lean language has a kernel that verifies that the logic is consistent. This is possible thanks to Lean's "functional" paradigm (relating to functions), which checks that objects have self-consistent types. In this framework, the library acts as an additional layer that redefines notation to a high degree.

Contents

1. Installation and Project Setup

You may install Lean to code on VScode by following the official guide.

1. **Create a project**: in VS Code, click the ∀ symbol in the top right and choose `New Project`. To work on an existing project, choose `Open Project`. Depending on internet speed, setup may take some time, but VS Code shows progress while the project is being prepared.
2. **Open the folder**: use VScode `File: Open Folder`. Explore the files to get a sense of project structure, but only edit the ones in the `My_project_name/` folder, as the others are managed automatically.
3. **Import Mathlib**: you can import the mathematical library by writing `import Mathlib` as the first line in the `Basic.lean` file.
4. **Lean4 InfoView**: when opening `.lean` files, you can use `Lean 4: InfoView: Toggle InfoView` to understand the intermediate state of a proof. Placing the cursor in the middle of the proof will show the available information and the necessary steps next. If strange errors appear but the proof is correct, you can press the `Restart File` button

Formalization projects start with a blueprint, which gets filled progressively: a sketch of the idea can be found on Terence Tao's blog, while you may find here a list of exemplars and the installation guide. For reference, this is an overview of the available theorems,

2. Type System

LEAN's language is based on the axioms of `Dependent Type Theory`. At a very high level, this allows for the existence of functions where the value of the first parameter can affect the type, and thus the value, of the later parameters that a function takes in. In this context, there is no difference between a function in the mathematical sense and in the programming sense.

To help visualize this change, consider the difference between the standard Cartesian Product and the type-dependent product:

- *Cartesian Product*: $\{(a, b) : a \in A, \quad b \in B\}$, here we combine two sets in a completely independent way.
- *Dependent Product*: $\{(a, b) : a \in A, \quad b \in B_a\}$, in this case the value of a directly affects the type, and thus possible values of b. Note that this assumes the existence of a type family $A \to \text{Type}$. The type of the second component depends on the value of the first.

This allows for functions to behave differently based on their inputs, but also to construct high level arguments by abstracting from the details. Just like in mathematics, where the same proof can be applied to all objects that share a specific algebraic structure, we can imagine a proof like a function that first takes as an argument the type of the object, followed by some fixed assumptions on the objects, which depend on the object, but may be imposed for all objects in a more general collection.

The second peculiarity of Lean's type system is the existence of an infinite hierarchy of universes. Propositions live in `Prop`, while data types live in `Type u` for some universe level `u`. In particular, `Type 0 : Type 1`, `Type 1 : Type 2`, and so on. This hierarchy prevents inconsistencies such as `Type : Type` and helps avoid paradoxes related to self-reference, such as Russell's paradox in naive set theory. Intuitively, collections of objects from one universe live in a higher universe, so a type is not itself an element of the same type. In practice, Lean infers universe levels automatically, assigning constructions to higher universes when needed.

For instance, consider the notion of a linear function. Since linearity can be defined for a wide class of objects, the same proof may be used on different spaces, by simply fixing the overarching space, and then defining a function on this space. This would be an example of type dependence.

For Mathlib formalizations, we must consider two fundamental objects:

- **Proposition**: object that expresses a statement, without imposing any assumption of it being true or false. They are constructed by combining basic facts through logic. For example, the statement `n = 2 → Even n` is itself a proposition. To help visualize this, consider the statement `n=2 => n even`, which is built by mapping statements `n=2` and `n even` through an implication function taking the form `imply : Statement x Statement -> Statement`. Since false statements may also be constructed in this way, thus we need another object to verify a statement.
- **Proof**: a term whose type is the proposition being proved. It is an object that represents a series of steps that results in the construction of a Proposition. In Mathlib, a proof is an object of type proposition, and it is at the same time a function that "transforms" some assumptions into a series of conclusions. Following the previous example, a proof that `n=2` implies `n even` would be an object of type "Statement that `n=2 => n even`", but it also may be equivalently formulated as a function that given an object of type `n=2` (that is a proof of the statement), constructs an object of type `n even`.

Proof Irrelevance: when verifying the correctness of a statement, the Lean language verifies that each intermediate step follows from the previous one, by simply checking that the types of the proofs are always compatible. In this fashion, all proofs of a statement are equivalent for the sake of verifying the statement.

Following this formalism, **axioms** act simply as an object that stands in for a proof, but whose definition is not explicitly stated, but is taken as a given. In fact, axioms take the role of lemmas during the intermediate steps in the logical chain of a proof. A similar notion is true in the case of hypotheses: any assumptions in the statement are equivalent in role to having a proof of that hypothesis. In brief, axioms and hypotheses provide terms that may be used as proofs of the corresponding propositions. This is natural, since for both of these statements we can use the fact they are an axiom or a hypothesis as a reason enough to treat them as true.

Another useful insight to this philosophy is in the case of proofs of implication. Suppose you have a proof that $a \wedge b \Rightarrow b \wedge c$. This is an inhabitant of the set $a \wedge b \Rightarrow b \wedge c$, but it can also be viewed as a function that converts a proof of statement $a \wedge b$ into a proof of type $b \wedge c$. In a sense, proofs of implication are bridges between different sets of proofs, whether they are assumptions, axioms or proofs that you have implemented yourself.

3. The Check Command

The exact definition of types may sometimes get confusing, since objects in Lean are capable of taking other objects and perform meta-programming changes on them. To navigate these issues, `#check obj` can be used to verify or infer types. This helps understand the language better, by identifying the meaning of a term, or to figure out what term is being introduced in a proof. In a similar fashion it is also possible to jump to the

```
variable (a b c : ℝ)

#check a
#check a + b
#check (a : ℝ)
#check mul_comm a b
#check mul_comm
#check (Type 20 → Type 21)
```

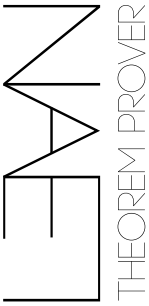
definition of a term in code with the shortcut `ctrl/cmd + click`.

The check command may be used anywhere, even in the middle of the statement of a proof.

4. Functional Syntax

The syntax of Mathlib is simple, but all-encompassing by using **syntax overloading**: unicode characters like $(\forall, \exists, \mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \leq, \neq, \in, \sum, \prod, \int \ldots)$ are used to represent complex concepts concisely. Most symbols can be inserted by simply writing "\" and the character shorthand, but more can be defined as needed. Moreover, hovering the mouse over a symbol will show the proper shorthand.

The fundamental unit of knowledge in Mathlib is that values have types. This is expressed using the `value : type` notation, which is used to construct all other complex functions. Consider the following definition of a "function":




```
theorem Complex.cos_add_sin_mul_I_pow (n : ℕ) (z : ℂ) :
  (cos z + sin z * I) ^ n = cos (↑n * z) + sin (↑n * z) * I
```

Here, the **theorem** keyword explains the definition, the name is `Complex.cos_add_sin_mul_I_pow`, and the type of the arguments is one variable `n` of type natural number and one variable `z` of type complex number, while the type of the output of the function is the statement of *De Moivre's theorem*. Observe also the presence of the `↑` character, which communicates upcasting: we convert the natural number n into a complex number, so that multiplication with a complex number is defined.

You may also encounter curly bracket arguments {...}, which fix the default values for the arguments, allowing them to be omitted, and square bracket arguments [...], which tell Lean to construct a type with some additional properties on the spot, typically for equipping parameters with some additional properties.

For example, the following is the definition of `Measurable`:

```
def Measurable [MeasurableSpace α] [MeasurableSpace β] (f : α → β) : Prop :=
  ∀ ⦃t : Set β⦄, MeasurableSet t → MeasurableSet (f ⁻¹' t)
```

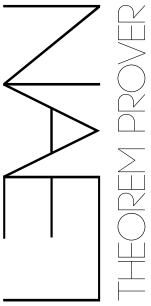
4.1. Namespaces

To avoid confusion for cases where the same name is used in different theories, `namespaces` are used, which consist of areas of code with a fixed set of names. To use names from a namespace without qualification, you can write open Foo, which affects name resolution in the current scope. By contrast, `import` loads another module and does not automatically open its namespaces. After defining a namespace, you can refer to its contents with qualified names such as `Foo.x`, open the namespace with `open Foo`, or introduce notation from the scope using `open scoped Foo`.

```
namespace Foo
  -- defines a new meaning for a unicode symbol in this scope
  scoped infix:68 " ≋ " => BEq.beq -- boolean equality

  def x := 1 -- we can define an object in a scope

  -- `x` is already accessible from outside as `Foo.x`
end Foo
```

After defining a namespace, we can access its contents by referring to the namespace, importing it, or just importing the notation.

```
-- we refer to them differently based on access rule
#check Foo.x

-- but we can also import the whole namespace
open Foo
-- now we do not need the "Foo." prefix
#check x

-- or just import the syntax rules (such as for unicode symbols)
open scoped Foo
#check (1 ≋ 2)
#check (· ≋ ·)
```



Namespaces become relevant when writing larger projects. We introduce them early to notify that they exist, but they will not be useful for some time.

5. Tactics

In order to lighten the burden of proving something one step at a time, Mathlib uses **meta-programming** extensively: there are algorithms that can quickly prove simple statements by taking the current state of the proof and attempting to construct a goal, which also gets indicated. This is very common with algebraic manipulations like equalities and inequalities, for which powerful tactics exist for rearranging terms within an expression. Tactics are introduced through the `by` keyword in the following way.

Some fundamental tactics, picked from the longer cheatsheet:

- `intro`: If the goal is `∀ x, P x` or `A → B`, introduces a new variable/hypothesis into the context (`intro x`, `intro h`), leaving the body as the new goal; supports patterns like `intro ⟨a, b⟩`.
- `unfold`: Expands a definition by replacing it with its body (`unfold f`, `unfold f g`); can target locations (`unfold f at h`).
- `rw`: Rewrites occurrences using equalities/iff (`rw [h]` left-to-right, `rw [← h]` right-to-left); can rewrite in hypotheses (`rw [h] at hx`) and chain rules (`rw [h1, h2]`).
- `ring`: Closes goals of the form `lhs = rhs` in commutative (semi)rings by normalizing both sides as polynomials in the ring operations; `ring!` is a more aggressive variant.
- `rel`: For relational goals like `≤` / `<`, transforms the goal using monotonicity/ congruence lemmas and the inequalities you provide; e.g. `rel [h1, h2]` reduces the goal to simpler subgoals.
- `have`: Adds a new local fact to context (`have h : P := t` or `have h : P := by ...`); supports destructuring patterns like `have ⟨h1, h2⟩ := h`.
- `exact`: Closes the goal by providing a term whose type is definitionally equal to the goal (`exact t`).
- `simp`: Simplifies the goal (or hypotheses) using simp lemmas and definitional reductions; common forms: `simp`, `simp [h1, h2]`, `simp at h`, `simp [*]`, `simp only [lemmas...]`.
- `linarith`: Solves goals that are linear equalities/inequalities by combining linear (in)equalities from the context (derives contradictions and bounds automatically).

6. Basic Tactics

We first consider an overview of the basic tactics which can be used to prove standard arithmetic statements or algebraic manipulations

rewrite

We can use `rw` to substitute the hypothesis into an equation. In the following example, we show a trivial implication of the form `a => b` by simply substituting in hypothesis.

```
example (a b : ℝ) (h : a = b) : a = b := by
  rw [h]
```

It is also possible to use lemmas directly, helping `ring` explore the correct direction. For example, in the following example, we can tell Mathlib to use the transformation $b * a = a * b$, using the `mul_comm` lemma.

```
example (a b c : ℝ) : a * b * c = b * (a * c) := by
  rw [mul_comm a b]    -- first obtain that b * a = a * b from a * b
  rw [mul_assoc b a c] -- apply associativity as (a * b) * c = b * (a * c)
```

Notably, we may also omit terms that are clear from context. This notational compactness is possible thanks to Mathlib's efficient type checking system, which connects the dots by looking at what types (statement proofs) are available and which are needed.

```
example (a b c : ℝ) : a * b * c = b * c * a := by
  rw [mul_assoc] -- apply substitution, but with the inverted hypothesis
  rw [mul_comm] -- apply substitution, but with the inverted hypothesis
```

For completeness, the definition of `mul_comm` is the following. It takes two elements of a Commutative Magma and produces the commute statement. Note how `u_1`, the universe of the magma's support (in the sense of hierarchy of types), and `G` are both inferred from context, which is why we do not provide them when invoking the statement.

```
mul_comm.{u_1} {G : Type u_1} [CommMagma G] (a b : G) : a * b = b * a
```

ring

The `ring` tactic helps to simplify an expression by performing simple operations like cancelling terms. It works by rearranging both sides of an expression to canonical normal form and verifying that they are equal.

```
example : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by
  ring -- can be simplified directly

example (a b c d : ℝ) (hyp : c = d * a + b) (hyp' : b = a * d)
    : c = 2 * a * d := by
  rw [hyp, hyp'] -- we simply apply both hypotheses
  ring -- simplify using the ring tactic
```

show

In a more nuanced way, sometimes we want to restate the current goal in a form that is easier to prove. We use `show` to replace the target by a more convenient one, which Lean accepts as the same goal.

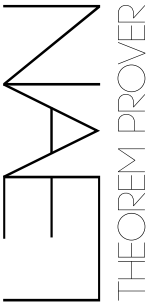
```
example (n : Nat) : n + 0 = n := by
  show n = n
  rfl -- reflexivity property of the "equality" relation
```

The tactic may also be used to make the target explicit before giving the proof term.

```
example (a b : ℝ) : a + b = b + a := by
  show a + b = b + a -- we first rearrange the goal
  exact add_comm a b -- this is the exact end goal, by just using add_comm
```

There is also a term-style version, which is guided by lemmas:

```
example (a b c : ℝ) : a * b * c = b * (a * c) := by
  show a * b * c = b * (a * c) from by -- statement rearranging
    rw [mul_comm a b, mul_assoc]       -- tactic that gets used
```



relation

We can use the `rel` tactic to apply an inequality which we have as a hypothesis to a statement. As before, this is a fundamental building block.

```
example {r s : ℚ} (h1 : s + 3 ≥ r) (h2 : s + r ≤ 3) : r ≤ 3 :=
  calc
    r = (s + r + r - s) / 2 := by ring
    _ ≤ (3 + (s + 3) - s) / 2 := by rel [h1, h2]
    _ = 3 := by ring
```

Exercises: First, Second, Third, Fourth

Use `norm_num` and `linarith [sq_nonneg (x - y)]` in exercises 3 and 4. You can use `#check` to understand how to use them.

apply

The `apply` tactic can be used to invoke a theorem in a specific setting. It also allows some generality on exact use. Also, we can use `·` to guide the subgoals, helping steer the tactic in the right direction.

```
-- transitivity of the relation "smaller or equal to"
#check (le_trans : a ≤ b → b ≤ c → a ≤ c)

example (x y z : ℝ) (h₀ : x ≤ y) (h₁ : y ≤ z) : x ≤ z := by
  apply le_trans -- steer the tactic in two steps:
  · apply h₀
  · apply h₁
```

linear arithmetic

The `linarith` tactic can handle basic linear arithmetic, such as filling in the steps required to simplify an equality or a basic inequality.

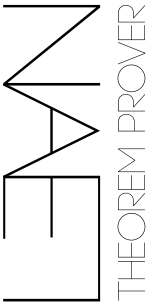
```
example (h : 2 * a ≤ 3 * b) (h' : 1 ≤ a) (h'' : d = 2) : d + a ≤ 5 * b := by
  linarith -- simple enough that it can be solved directly
```

6.1. Reverse Directions

This is not a tactic, but it is commonly used: some relations like equality may have to be applied applied in reverse order. This requires the `←` symbol, which is written with `\<-`.

```
example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b :=
  calc
    (a + b) * (a + b) = a * a + b * a + (a * b + b * b) := by
      rw [mul_add, add_mul, add_mul]
    _ = a * a + (b * a + a * b) + b * b := by
      rw [← add_assoc, add_assoc (a * a)] -- reverse associativity
    _ = a * a + 2 * (a * b) + b * b := by
      rw [mul_comm b a, ← two_mul] -- reverse order
```

Listing 1: A more complex example using a couple of theorems



Exercises: First, Second

7. The Variable Keyword

The `variable` keyword is used to define new objects, so they can be reused multiple times over different theorems or examples.

```
variable (x y z : ℝ)

-- these examples use x, y, z from the `variable` command above.

example : x + y = y + x := by
  sorry

example : (x + y) + z = x + (y + z) := by
  sorry

example : x ^ 2 >= 0 := by
  sorry

example (h : x = y) : x^2 = y^2 := by
  sorry
```

8. Searching for Tactics

To help navigate Mathlib's multitude of theorems and tactics, there are many resources that can be used to identify their exact names:

- `apply?` : the InfoView panel can suggestion an appropriate tactic.

```
example : 0 ≤ a ^ 2 := by
  apply?
```

⟹

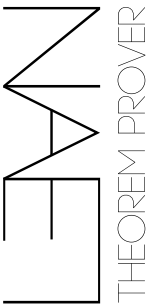
```
Try this:
  [apply] exact sq_nonneg a
```

- Guessing: Lean4 naming conventions help maintain consistency across libraries, allowing users to guess exact theorem names.
- `Search Engines` : Lean Explore and Moogle provide semantic search capabilities.
- `Zulip` : the official Lean forum, provides users with a `Is there code for X?` page.

9. Calc Mode

When we write `calc` after the `by` keyword, we are switching to a step-by-step derivation which allows us to concatenate expressions with justifications in a more readable way. Each line proves that the current expression is related to the previous one, and Lean combines the lines using transitivity. It is mainly used for equalities, inequalities, and other transitive relations

We should now have enough of an understanding to read a more complex proof, and understand it to some extent. The following example proves that the affine combination between Lotteries (a probability distribution over a countable support X) has probability that sums to 1.




```
private lemma mix_sum_one (p q : Lottery X) (α : Real) :
    -- sum over the support of X, where x is the index
    ∑ x, (α * p.val x + (1 - α) * q.val x) = 1 := by

  -- use the calc tactic to prove by direct simplification
  calc
    -- rewrite the left side, and reach the right side
    ∑ x, (α * p.val x + (1 - α) * q.val x)

    -- the new goal is written on the next line,
    -- but the justification comes from the tactic after `by`
    _ = α * ∑ x, p.val x + (1 - α) * ∑ x, q.val x := by

        -- the rewrite tactic, which uses `Finset` properties
        rw [Finset.sum_add_distrib, Finset.mul_sum, Finset.mul_sum]

    -- use rewrite again with the properties of `p` and `q` Lotteries
    _ = α * 1 + (1 - α) * 1 := by rw [p.property.2, q.property.2]

    -- use the `ring` tactic to do simple algebraic cancellations
    _ = 1 := by ring
```

Listing 2: von Neumann-Morgenstern Utility Theorem - lemma

10. Control Flow Tactics

Some tactics involve the manipulation of the proof step, being used to divide the proofs into cases, introduce intermediate steps, or conclude a proof.

exact

The `exact` tactic can be used to directly tell Lean that we have reached our conclusion. With this keyword, we explicitly communicate that we have reached our goal and the proof is complete.

```
example (P Q : Prop) (hp : P) (hq : Q) : P := by
  exact hp -- closes the goal because hp is exactly of type P

-- using exact with a function application
example (n : ℕ) (h : n = 5) : n + 0 = 5 := by
  rw [add_zero] -- simplify n + 0 to n
  exact h       -- the goal is now n = 5, which matches h exactly
```

It is also possible to "guide" the tactic by providing lemmas to it: in the following example, the lemma `sq_pos_of_pos` produces a proof that the square is also positive, from a proof that the number is positive.

```
example (a : ℝ) (ha : 0 < a) : 0 < a^2 := by
  exact sq_pos_of_pos ha
```

10.1. Changing Goal

When writing a proof, the Lean `InfoView` keeps track of the all the statements that have been introduced so far and the goal that you are trying to prove. The last step connects the current state of the proof to a goal with one tactic; clearly, the proof is complete and minimal if it links every assumptions to the final goal.

When constructing more complex proofs, it may be necessary to change the current goal for a variety of reasons: proving a simpler statement first, simplifying the goal or proving an equality in two steps. The following proofs are useful in such cases.

have

We can use the `have` tactic to introduce an intermediate goal, that is, an additional hypothesis, in the middle of the proof.

```
example {a b : ℝ} : 2 * a * b ≤ a^2 + b^2 := by
  have h : 0 ≤ a^2 - 2 * a * b + b^2 := -- define the new hypothesis
    calc                                -- prove it with calc
      0 ≤ (a - b)^2 := by apply pow_two_nonneg
      _ = a^2 - 2 * a * b + b^2 := by ring

  linarith -- close the goal with linarith
```

More powerful tactics also exists, with the tradeoff that greater automation results in slower verification. For this reason, powerful tactics are often replaced with more precise solutions in the foundational theorems in Mathlib.

Exercises: First, Second, Third, Fourth

apply

The `apply` keyword may be used to act on the goal by reducing it to a simpler form. The following example is more involved: we first prove as a subgoal that $a^2 > 0$, which follows from the $a > 0$ assumption; we apply the same lemma, relying on this fact, completing the proof.

```
example (a : ℝ) (ha : 0 < a) : 0 < (a^2)^2 := by
  have h2 : 0 < a^2 := by     -- we declare `0 < a^2` as a subgoal
    apply sq_pos_of_pos  -- we start proving the subgoal
    exact ha             -- concludes the subgoal
  exact sq_pos_of_pos h2 -- we finished the subgoal, and now we prove the main
goal using it.
```

Exercises: First

intro

To prove implications, of the form $A \Rightarrow B$, one starts by assuming A and proceeds with obtaining the statement B. The `intro` tactic is used in these case: it takes an assumption from the goal and moves it into the local context.

```
-- Using intro to handle implications
example (P Q : Prop) : P → Q → P := by
  intro hP -- Moves 'P' into the context as hypothesis 'hP'
  intro hQ -- Moves 'Q' into the context as hypothesis 'hQ'
  exact hP -- The goal is now 'P', which we have

-- Using intro for universal quantification
example : ∀ (n : ℕ), n + 0 = n := by
  intro n  -- Picks an arbitrary natural number 'n'
  rw [add_zero]
```


Another delicate example is the following, where we prove a logical implication. To read the proof, remember that in Lean, `f x` means applying `f` to `x`, and `f x y` means `(f x) y`. Also, implication is right-associative, so `p → q → r` means `p → (q → r)`. In other words, it is a function that takes a proof of `p` and returns a function that takes a proof of `q` and returns a proof of `r`.

```
example (p q r : Prop) : (p → q) → (p → q → r) → p → r := by
  -- h1 : p → q, h2 : p → q → r, h3 : p
  intro h1 h2 h3

  -- From h1 and h3, obtain a proof of q.
  have hq : q := h1 h3

  -- From h2, h3, and hq, obtain a proof of r.
  -- First, h2 h3 : q → r
  -- Then, (h2 h3) hq : r
  have hr : r := h2 h3 hq

  exact hr
```

constructor

In order to prove statements with a direct and converse proof, we can use the `constructor` keyword in the following way: we use the scope of a constructor, then prove each implication direction by first introducing the assumption with the intro keyword. We also use the bullet point `·` character to focus the current goal, which hides the final goal.

```
example (a b : ℝ) : (a-b)*(a+b) = 0 ↔ a^2 = b^2 := by
  constructor
  · intro h -- h : (a - b) * (a + b) = 0
    calc
      a ^ 2 = b^2 + (a - b) * (a + b)  := by sorry
          _ = b^2 + 0                  := by sorry
          _ = b^2                      := by sorry
  · intro h -- h : a ^ 2 = b ^ 2
    calc
      (a-b)*(a+b) = a^2 - b^2  := by sorry
                _ = b^2 - b^2  := by sorry
                _ = 0          := by sorry
```

In this case, Lean helps us by dividing the proof into the two canonical directions of implication, so we can prove one at a time. At the end the `constructor` packs the two sub-proofs into a single combined structure, that is, the *iff* statement.

10.2. Equivalences

On a related note, there are simpler ways than the `constructor` tactic to prove equivalences between statements: one can rewrite an equality to obtain a trivially true equation, such as `1 = 1`.

```
-- `sub_nonneg : 0 ≤ y - x ↔ x ≤ y`
example {a b c : ℝ} : c + a ≤ c + b ↔ a ≤ b := by
  rw [← sub_nonneg] -- `rw` takes an equivalence
  have key : (c + b) - (c + a) = b - a := by
    ring
  rw [key] -- `rw` applies the equality `key`
  rw [sub_nonneg] -- switch back to reach the tautology a ≤ b ↔ a ≤ b
```

Once you have an *iff* statement, you can invoke one of its directions of implication with the `name.1` or `name.2` syntax.

```
-- `add_le_add_iff_right (c : ℝ) : a + c ≤ b + c ↔ a ≤ b`
example {a b : ℝ}  (ha : 0 ≤ a) : b ≤ a + b := by
  calc
    b = 0 + b := by ring
    _ ≤ a + b := by exact (add_le_add_iff_right b).2 ha
```

11. Additional References

Lean may be used online, with no installation at this link.

11.1. Sources

Many examples were taken from the "Mathematics in Lean" and "Glimpse of Lean". They are excellent resources for more in-depth study.

- *Glimpse of Lean*, Patric Massot, Online Repository
- *Mathematics in Lean*, leanprover-community, Online Book
- *Lean 4 Tactic Cheatsheet*, Floris van Doorn, Source
- *The Mechanics of Proof*, Heather Macbeth. Online Book
- *Mechanized Proofs of the vNM Utility Theorem*, Jingyuan Li, Code, Paper
- *Github Lean Project Template*, leanprover-community, LeanProject Template

11.2. Complete Projects

- Analytic Number Theory Exponent Database by Terence Tao et al.
- Equational Theories by Terence Tao et al.
- Polynomial Freiman Ruzsa Conjecture by Terence Tao et al.
- Infinity Cosmos by Emily Riehl et al.
- ABC Exceptions by Bhavik Mehta et al.
- Proofs from THE BOOK by Moritz Firsching et al.
- Soundness of FRI by Bolton Bailey et al.
- Sphere Packing in 8 Dimensions by Maryna Viazovska et al.
- Groupoid Model of Homotopy Type Theory by Sina Hazratpour et al.
- Weil's Converse Theorem by Chris Birkbeck et al.
- Dirichlet Nonvanishing by Chris Birkbeck et al.
- Spectral Theorem by Oliver Butterley and Yoh Tanimoto.
- Automata Theory by Stefan Hetzl et al.
- Seymour's Decomposition Theorem by Ivan Sergeyev et al.
- NeuralNetworks by Matteo Cipollina.

11.3. Smaller Projects

- *Unit Fractions Density Conjecture*: git, paper, blueprint
- *Liquid Tensor*: git, paper and image, blueprint.
- *Arithmetic Progressions - Almost Periodicity*: home, documentation, git
- *Brauer Group and Galois Cohomology*: paper, code documentation, blueprint
- *Brownian Motion*: git, documentation, blueprint,
- *Toric Varieties*: home, paper without lean references, blueprint, upstream-ready
- *Combinatorial Games*: home, git,
- *Forbidden Matrix Theory*: overview, git, blueprint code.
- *∞-Cosmoi for Lean*: home, blueprint, docs

Related but through a different formalization language: Busy Beaver Challenge.